REG NO - 24BEE0211, 24BEE0213 & 24BEE0216 DATE- 22/10/2026
NAME- SANKALP SINGH BAIS, SWASTIKA CHAKRAVORTY & JESLIN ELSA JOSE SLOT- L51+L52
FACULTY- DR. AMUTHA PRABHA N EXPERIMENT NO- 10

Synchronizing Real-Time Face Tracking with Directional Audio Filtering

AIM:

To design and implement an Audio-Visual Zoom (AV Zoom) system that synchronizes real-time computer vision face tracking with dynamic digital signal processing (DSP) audio beamforming to spatially isolate a moving speaker's voice, and to analyze the synchronization using data visualization.

APPARATUS REQUIRED:

Software: MATLAB, Computer Vision Toolbox.
Hardware: Personal Computer.
Input Data: A standard `.mp4` video file containing a true stereo audio track (2 distinct channels).

ALGORITHM:

1. **Initialization:** Define environmental parameters, including the speed of sound (343 m/s), microphone spacing (0.08 m), and the camera's field of view (70 degrees). Initialize data arrays to store tracking angles and audio delays for graphing.
2. **Data Ingestion:** Load the input .mp4 video and extract its stereo audio track. Calculate the corresponding audio chunk size per video frame to ensure precise temporal alignment.
3. **Object Setup:** Instantiate a vision.CascadeObjectDetector for facial tracking and a vision.VideoFileWriter to multiplex the final processed video and audio into a single .avi file.
4. **Processing Loop:** For every frame in the video, perform the following steps:
    - **Capture:** Read the current video frame and its corresponding audio chunk.
    - **Visual Tracking:** Pass the frame through the face detector to obtain the bounding box of the speaker's face.
    - **DoA Calculation:** Calculate the center pixel coordinate of the face and map it to a physical Direction of Arrival (DoA) angle relative to the camera's center axis.
    - **Visual Zoom:** Crop the video frame around the face's bounding box and resize it to the original resolution.
    - **TDOA Calculation:** Using the DoA angle, calculate the Time Difference of Arrival (TDOA) of sound between the two microphones, and convert this time delay into discrete audio samples.
    - **Audio Beamforming:** Apply Delay-and-Sum (DAS) beamforming by shifting the phase of either the left or right audio channel by the calculated sample delay, then average the two channels.
    - **Multiplexing & Logging:** Write the dynamically cropped video frame and the spatially filtered audio chunk simultaneously to the output file, and log the current angle and delay values.
5. **Termination & Analysis:** Release the video writer resources and generate plotted graphs displaying the tracked visual angle and applied DSP delay over the duration of the sequence.

MATLAB PROGRAM:

```
% AV Zoom: Complete Audio-Visual Tracking, Beamforming, and Analysis
% Outputs a single AVI file with both video and audio, and generates graphs.

%% 1. System Parameters & Setup
inputFile = 'C:\Users\swast\Documents\dsp10video.mp4';
outputFile = 'C:\Users\swast\Documents\Final_AV_Zoom.avi';

c = 343;                    % Speed of sound (m/s)
micSpacing = 0.08;          % Distance between mics (8 cm)
cameraFOV = 70;             % Camera Field of View in degrees
zoomScale = 0.5;            % 50% zoom crop factor

disp('Initializing AV Zoom System...');

% Setup Video and Audio Readers
vidReader = VideoReader(inputFile);
[audioIn, fs] = audioread(inputFile);
frameRate = vidReader.FrameRate;
frameWidth = vidReader.Width;
frameHeight = vidReader.Height;

% Verify stereo audio
if size(audioIn, 2) < 2
    error('The input video must contain a stereo audio track (2 channels).');
end
```

```
% Setup Face Detector
faceDetector = vision.CascadeObjectDetector('MergeThreshold', 7);

% Setup Video/Audio Muxing Writer (AVI format allows combined AV writing)
vidWriter = vision.VideoFileWriter(outputFile, ...
    'FileFormat', 'AVI', ...
    'AudioInputPort', true, ...
    'FrameRate', frameRate);

% Calculate chunk sizes for audio based on video framerate
samplesPerFrame = floor(fs / frameRate);
totalFrames = floor(size(audioIn, 1) / samplesPerFrame);

% Arrays to store data for graphing
angleHistory = zeros(totalFrames, 1);
delayHistory = zeros(totalFrames, 1);

disp('Starting AV Zoom Loop (Writing to single AVI)...');

%% 2. The Real-Time Synchronization Loop
for frameIdx = 1:totalFrames

    if ~hasFrame(vidReader)
        break;
    end

    % --- A. CAPTURE ---
    originalFrame = readFrame(vidReader);
    startSample = (frameIdx - 1) * samplesPerFrame + 1;
    endSample = frameIdx * samplesPerFrame;
    audioChunk = audioIn(startSample:endSample, 1:2);

    % --- B. VISUAL TARGETING (Computer Vision) ---
    bbox = step(faceDetector, originalFrame);

    if ~isempty(bbox)
        faceBox = bbox(1, :); % Grab the first detected face
        faceCenterX = faceBox(1) + (faceBox(3) / 2);
        faceCenterY = faceBox(2) + (faceBox(4) / 2);

        % Calculate DoA Angle
        normalizedX = (faceCenterX - (frameWidth / 2)) / (frameWidth / 2);
        targetAngleDeg = normalizedX * (cameraFOV / 2);

        % Define crop region for visual zoom
        cropW = frameWidth * zoomScale;
        cropH = frameHeight * zoomScale;
        cropX = max(1, faceCenterX - (cropW / 2));
        cropY = max(1, faceCenterY - (cropH / 2));
        cropRect = [cropX, cropY, cropW, cropH];
    else
        % Fallback if face is lost
        targetAngleDeg = 0;
        cropRect = [(frameWidth*(1-zoomScale))/2, (frameHeight*(1-zoomScale))/2, ...
                    frameWidth*zoomScale, frameHeight*zoomScale];
    end

    % Apply visual crop and resize back to original resolution
    zoomedFrame = imcrop(originalFrame, cropRect);
    finalFrame = imresize(zoomedFrame, [frameHeight, frameWidth]);

    % --- C. AUDIO BEAMFORMING (DSP) ---
    targetAngleRad = deg2rad(targetAngleDeg);
    timeDelay = (micSpacing * sin(targetAngleRad)) / c;
    sampleDelay = round(timeDelay * fs);

    % Store data for graphs
    angleHistory(frameIdx) = targetAngleDeg;
    delayHistory(frameIdx) = sampleDelay;

    zoomedAudio = zeros(samplesPerFrame, 1);
    if sampleDelay > 0
        shiftedMic1 = [zeros(sampleDelay, 1); audioChunk(1:end-sampleDelay, 1)];
        zoomedAudio = (shiftedMic1 + audioChunk(:, 2)) / 2;
    elseif sampleDelay < 0
        sampleDelay = abs(sampleDelay);
        shiftedMic2 = [zeros(sampleDelay, 1); audioChunk(1:end-sampleDelay, 2)];
        zoomedAudio = (audioChunk(:, 1) + shiftedMic2) / 2;
    else
        zoomedAudio = (audioChunk(:, 1) + audioChunk(:, 2)) / 2;
    end

    % --- D. WRITE COMBINED OUTPUT ---
    % Write both the visual frame and the audio chunk at the exact same time
    step(vidWriter, finalFrame, zoomedAudio);

    if mod(frameIdx, 30) == 0
        fprintf('Processed Frame %d/%d (Angle: %0.1f deg)\n', frameIdx, totalFrames, targetAngleDeg);
    end
end

%% 3. Finalize and Save
release(vidWriter);
disp('Processing Complete!');
disp(['Combined AV file saved to: ', outputFile]);
```

```
%% 4. Generate System Analysis Graphs
figure('Name', 'AV Zoom System Analysis', 'Position', [100, 100, 800, 600]);

% Graph 1: Visual Tracking Angle
subplot(2,1,1);
plot(1:totalFrames, angleHistory, 'b-', 'LineWidth', 1.5);
title('Computer Vision: Tracked Face Angle (Direction of Arrival)');
xlabel('Frame Number');
ylabel('Angle (Degrees)');
grid on;

% Graph 2: DSP Audio Delay
subplot(2,1,2);
plot(1:totalFrames, delayHistory, 'r-', 'LineWidth', 1.5);
title('DSP: Dynamic Audio Sample Delay Applied');
xlabel('Frame Number');
ylabel('Delay (Samples)');
grid on;
```

OUTPUT:



Fig. 1: Frame from the input video showing both speakers.



Fig. 2: Frame from the output Final_AV_Zoom.avi, dynamically cropped around the tracked speaker and resized to the original resolution.

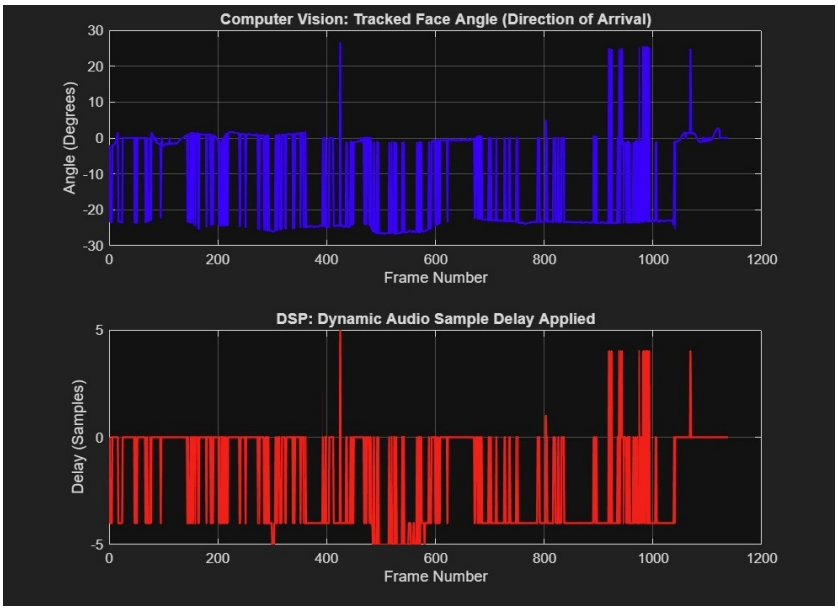


Fig. 3: System analysis graphs: tracked face angle (Direction of Arrival) and the dynamic audio sample delay applied per frame.

RESULT

The Audio-Visual Zoom program processed the prerecorded stereo input video frame by frame and generated a single, synchronized .avi output file. Each output frame was dynamically cropped around the tracked subject and resized to the original resolution, while the matching audio chunk was shifted by the computed sample delay and averaged across the two channels. The logged histories (Fig. 3) show the applied integer sample delay following the tracked face angle over the full sequence.

INFERENCE

The experiment demonstrates that visual face-tracking data can act as the control signal for a delay-and-sum audio beamformer when processing prerecorded media frame by frame. The direction-of-arrival angle estimated from the tracked face position determines the time delay applied between the two audio channels, so the beamformer's steering direction follows the moving subject through the sequence. The recorded angle and delay histories confirm that the visual tracking and the audio steering remain synchronized throughout the processing loop, with the achievable directional emphasis set by the spacing of the two recorded channels.